Shyamoli Textile Engineering College
Department of Computer Science and Engineering
CSE-4237: Digital Image Processing

Lec-18: Hadamard Transform

Digital Image Processing by Rafael C. Gonzalez and Richard Eugene Woods (Chapter-7)

Md. Ariful Islam
Assistant Professor
Dept. of Robotics and Mechatronics Engineering
University of Dhaka

2. Hadamard Transform

The Hadamard transform is *based on the Hadamard matrix*, which is a *square matrix* of size $2^n \times 2^n$ (where n is a positive integer).

The matrix *consists of only +1 and -1 values*, and its *rows (or columns) are orthogonal to each other*.

In linear algebra, two vectors (such as rows or columns of a matrix) are orthogonal if their dot product is zero. That means they are perpendicular in an n-dimensional space.

The transform *decomposes an image or signal into a set of basis functions*, similar to how the Fourier transform decomposes a signal *into sine and cosine components.*

2.1 Visualization of Basis Functions

For a 2×2 Hadamard transform, the basis functions are:

Basis 1 (DC component): $\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$

Represents *the average intensity of the image*.

Basis 2 (Horizontal variation): $\begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix}$

Represents *horizontal edges or patterns*.

Basis 3 (Vertical variation): $\begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix}$

Represents *vertical edges or patterns*.

Basis 4 (Diagonal variation): $\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$

Represents *diagonal edges or patterns*.

2.2 Key Properties of the Hadamard Transform

Orthogonality: The rows and columns of the Hadamard matrix are orthogonal, meaning the *transform preserves energy and is invertible*.

A matrix is invertible if there exists another matrix (called its inverse) that, when multiplied with the original matrix, results in the identity matrix.

$$A \cdot A^{-1} = A^{-1} \cdot A = I_n$$

Symmetry: The *Hadamard matrix is symmetric*, so $H = H^T$.

Real-Valued: Unlike the Fourier transform, the Hadamard transform *operates on real-valued data*, making it simpler to implement.

2.3 How the Hadamard Transform Works

Hadamard Matrix Construction

The Hadamard matrix H_n of size $2^n \times 2^n$ can be constructed recursively:

$$H_1 = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad H_n = \begin{bmatrix} H_{n-1} & H_{n-1} \\ H_{n-1} & -H_{n-1} \end{bmatrix}$$

$$H_2 = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}$$

Applying the Transform

For an image I of size $2^n \times 2^n$, the Hadamard transform T is computed as:

$$T = H_n \cdot I \cdot H_n$$

Here, H_n is the Hadamard matrix, and I is the image matrix.

Inverse Transform

The inverse Hadamard transform is simply: $$I = \frac{1}{2^n} H_n \cdot T \cdot H_n$$

The factor $\frac{1}{2^n}$ normalizes the result.

Example: Let's assume we have a 2×2 grayscale image represented as a matrix I:

$$I = \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix}$$

Here, the pixel values are:

I[0,0]=10

I[0,1]=20

I[1,0]=30

I[1,1]=40

Step 2: Define the Hadamard Matrix

For a 2×2 image, the Hadamard matrix H_2 is:

$$H_2 = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

Step 3: Apply the Hadamard Transform

The Hadamard transform T of the image *I* is computed as:

$$T = H_2 \cdot I \cdot H_2$$

Step 3.1: Compute $H_2 \cdot I$

Multiply the Hadamard matrix H_2 with the image matrix *I*:

$$H_2 \cdot I = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \cdot \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix}$$

Perform the matrix multiplication:

$$H_2 \cdot I = \begin{bmatrix} (1 \cdot 10 + 1 \cdot 30) & (1 \cdot 20 + 1 \cdot 40) \\ (1 \cdot 10 + (-1) \cdot 30) & (1 \cdot 20 + (-1) \cdot 40) \end{bmatrix}$$

$$H_2 \cdot I = \begin{bmatrix} 40 & 60 \\ -20 & -20 \end{bmatrix}$$

Step 3.2: Compute $(H_2 \cdot I) \cdot H_2$

Now multiply the result from Step 3.1 with H_2 again:

$$(H_2 \cdot I) \cdot H_2 = \begin{bmatrix} 40 & 60 \\ -20 & -20 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

Perform the matrix multiplication:

$$(H_2 \cdot I) \cdot H_2 = \begin{bmatrix} (40 \cdot 1 + 60 \cdot 1) & (40 \cdot 1 + 60 \cdot (-1)) \\ (-20 \cdot 1 + (-20) \cdot 1) & (-20 \cdot 1 + (-20) \cdot (-1)) \end{bmatrix}$$

$$(H_2 \cdot I) \cdot H_2 = \begin{bmatrix} 100 & -20 \\ -40 & 0 \end{bmatrix}$$

So, the transformed image T is:

$$T = \begin{bmatrix} 100 & -20 \\ -40 & 0 \end{bmatrix}$$

Interpretation of the Transformed Image:

$$T = \begin{bmatrix} 100 & -20 \\ -40 & 0 \end{bmatrix}$$

Top-left coefficient (100):

This is the **DC component** (average intensity) of the image. It *represents the overall brightness or intensity level of the image*.

Other coefficients (-20, -40, 0):

These are the **AC components**, which represent the variations or details in the image. Specifically:

- The *negative values indicate that certain patterns (edges, textures, etc.) are present* in the image.
- A *value of 0 means that the corresponding basis function does not contribute to the image*.

Step 4: Inverse Hadamard Transform

To reconstruct the original image, we *apply the inverse Hadamard transform*. The inverse transform is given by:

$$I = \frac{1}{2^n} H_2 \cdot T \cdot H_2$$

For a 2×2 image, $n = 1$, so $2^n = 2$. Thus:

$$I = \frac{1}{2} H_2 \cdot T \cdot H_2$$

Step 4.1: Compute $H_2 \cdot T$

Multiply H_2 with T:

$$H_2 \cdot T = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \cdot \begin{bmatrix} 100 & -20 \\ -40 & 0 \end{bmatrix}$$

Perform the matrix multiplication:

$$H_2 \cdot T = \begin{bmatrix} (1 \cdot 100 + 1 \cdot (-40)) & (1 \cdot (-20) + 1 \cdot 0) \\ (1 \cdot 100 + (-1) \cdot (-40)) & (1 \cdot (-20) + (-1) \cdot 0) \end{bmatrix}$$

$$H_2 \cdot T = \begin{bmatrix} 60 & -20 \\ 140 & -20 \end{bmatrix}$$

Step 4.2: Compute $(H_2 \cdot T) \cdot H_2$

Now multiply the result from Step 4.1 with H_2:

$$(H_2 \cdot T) \cdot H_2 = \begin{bmatrix} 60 & -20 \\ 140 & -20 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

Perform the matrix multiplication:

$$(H_2 \cdot T) \cdot H_2 = \begin{bmatrix} (60 \cdot 1 + (-20) \cdot 1) & (60 \cdot 1 + (-20) \cdot (-1)) \\ (140 \cdot 1 + (-20) \cdot 1) & (140 \cdot 1 + (-20) \cdot (-1)) \end{bmatrix}$$

$$(H_2 \cdot T) \cdot H_2 = \begin{bmatrix} 40 & 80 \\ 120 & 160 \end{bmatrix}$$

Step 4.3: Normalize the Result

Finally, multiply by 1/2 to normalize:

$$I = \frac{1}{2}\begin{bmatrix} 40 & 80 \\ 120 & 160 \end{bmatrix} = \begin{bmatrix} 20 & 40 \\ 60 & 80 \end{bmatrix}$$

To recover the exact original image, you would need to divide the reconstructed image by 2.

This confirms that the Hadamard transform is lossless and perfectly reversible.

2.3 Applications in Image Processing

Image Compression:

The Hadamard transform can be used to compress images by *discarding small coefficients in the transformed domain*, similar to JPEG compression using the Discrete Cosine Transform (DCT).

Feature Extraction:

The transform can highlight certain features in an image, such as *edges or textures, by analyzing the coefficients in the transformed domain*.

Pattern Recognition:

The *Hadamard transform can be used to extract patterns or signatures from images*, which can then be used for *classification or matching*.

Noise Reduction:

By thresholding small coefficients in the transformed domain, noise can be reduced while preserving important image features.